BACKEND REFERENCE

SpeakTwin

Everything in the backend — libraries, models, algorithms, and how to explain them

Version 1.2.0

Backend 35 Python modules · ~5,300 lines

Models 10 (1 required, 9 optional)

Tests 196 passing

Author Rohan Untawale

Contents

1 · What the Backend Does

The backend receives an audio recording of someone speaking, works out *how* they spoke — pace, loudness, vocal variety, filler words, clarity — and returns coaching. Optionally it also receives body-position landmarks from the browser and scores posture and gesture.

It is a **FastAPI** service. The core pipeline is deliberately classical signal processing plus one speech-recognition model, so it runs on any CPU with no GPU. Nine further neural models are available but **switched off by default**: the base install must stay small and must work.

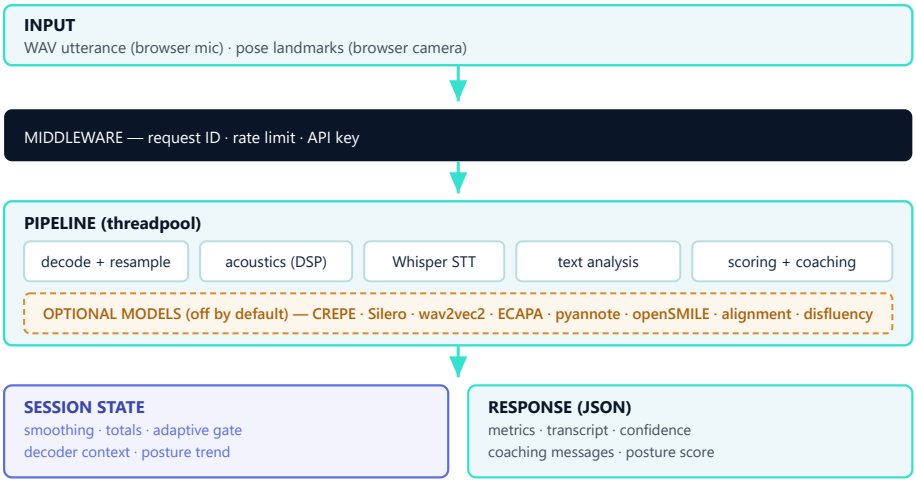

Figure 1 — Backend data flow. Dashed band is optional and disabled unless enabled explicitly.

1.1 The four constraints that explain every decision

Constraint	What it forced
Must run without a GPU	Classical DSP for pitch/energy/pauses; int8-quantised Whisper; all neural models optional
Must run on a server	No microphone or camera on the server — both are captured in the browser and sent as data
Near-real-time	Blocking work off the event loop; every model has a measured time budget
Must degrade, never break	Cloud STT, the LLM, and all nine optional models fail to a working fallback

2 · The Stack

2.1 Required — the base install

This is everything `pip install -r requirements.txt` brings in. The service is fully functional with only these.

Library	Version	What it does here
FastAPI	0.141.1	Web framework. Chosen for native async, Pydantic validation, and automatic OpenAPI docs at `/docs`
Uvicorn	0.52.1	ASGI server that actually runs the app
Pydantic	2.13.4	Every response is a typed model — this is what makes the API a contract rather than a guess
python-multipart	0.0.32	Parses the multipart upload carrying the WAV
NumPy	1.26.4	All array maths: RMS, framing, statistics
SciPy	1.17.1	Two specific things: `resample_poly` for exact sample-rate conversion, and FFT `correlate` for pitch
SoundFile	0.14.0	Audio decoding via libsndfile — WAV, FLAC, OGG at any sample rate
faster-whisper	1.2.1	Speech recognition. A CTranslate2 reimplementation of Whisper, ~4x faster than the reference
CTranslate2	4.8.1	The quantised inference engine underneath. int8 is what makes CPU Whisper viable
onnxruntime	1.28.0	Runs the Silero VAD that trims silence before decoding
python-dotenv	1.2.2	Loads `.env`, once, from the project root
groq / openai	lazy	Cloud STT and LLM clients. Imported only if a key is configured, so they are optional at runtime

> **POINT WORTH MAKING**
>
> The dependency list is deliberately short for what the app does. Pitch tracking, pause detection, and loudness are ~250 lines of NumPy and SciPy rather than a library — which is why **librosa was removed**. It pulls in numba and ~50 MB to provide functions that were replaced by code you can read and test.

2.2 Optional — the ML extras

`requirements-ml.txt`. Adds roughly 700 MB; nothing here is needed for the app to work.

Library	Version	What it does here
PyTorch	2.13.0+cpu	Runtime for every neural model in the optional layer
torchaudio	2.11.0+cpu	Audio tensor handling
Transformers	5.15.0	Loads the Hugging Face models (emotion, disfluency backbone)
torchcrepe	0.0.24	CREPE neural pitch tracker
SpeechBrain	1.1.0	ECAPA-TDNN speaker embeddings
openSMILE	2.6.0	eGeMAPS prosody features. Not a neural model — a compiled DSP toolkit
pyannote.audio	optional	Speaker diarization. Gated model, needs a Hugging Face token
datasets	5.0.1	Corpus loading for training
scikit-learn	1.9.0	Fitting the confidence weights against human ratings

2.3 Total footprint

Bundle	Size
Base install (everything in 2.1)	324 MB
With the ML extras (2.2)	~1 GB

3 · The Models

Ten models. One is required; nine are optional and off by default. Every latency below was measured on this machine — CPU only, no GPU, after warm-up.

Model	Type	Where	Cost	Job
Whisper base.en	Encoder–decoder transformer (~74M)	Server	~2.6 s / utt	**Speech to text.** The only required model
Whisper large-v3	Transformer (~1.5B)	Groq / OpenAI cloud	~1 s	Same job, far better on accents. Needs an API key
CREPE tiny	CNN pitch tracker	Server	~780 ms	Fundamental frequency — beats autocorrelation on real speech
Silero VAD	Small recurrent net	Server	~270 ms	Tells speech from noise. Replaces the energy gate
wav2vec2-base SUPERB-ER	Self-supervised + classifier (~95M)	Server	~385 ms	Emotion / vocal tension from the waveform
ECAPA-TDNN	Time-delay CNN	Server	~375 ms	192-dim speaker embedding — is it still the same person?
pyannote 3.1	Segmentation + embedding pipeline	Server	session-level	Who spoke when. Gated model
openSMILE eGeMAPS	**Not neural** — DSP toolkit	Server	~1600 ms	88 prosody features: jitter, shimmer, HNR
WavLM base+	Self-supervised (~94M)	Training only	—	Backbone to fine-tune the acoustic filler detector
MediaPipe Pose Lite	CNN landmark detector	**Browser**	~30 fps	33 body landmarks for posture and gesture

> **THE SINGLE MOST IMPORTANT SLIDE-WORTHY FACT**
>
> The per-chunk models sum to roughly **3.4 seconds against a 2.5-second budget**. You cannot enable them all on a CPU. The pairing that earns its cost is **CREPE + Silero (~1.05 s, 42% of budget)**. openSMILE at 1.6 s belongs offline, building training features, not on the live path.

3.1 Why each optional model exists

CREPE — neural pitch

Autocorrelation is cheap and works on clean tones, but real speech is breathy and noisy and produces octave errors. CREPE is a CNN trained to read f0 directly from the waveform and is markedly more reliable. The DSP tracker stays wired as the fallback, so pitch is always produced.

Silero VAD — is this actually speech?

An energy threshold cannot tell a quiet speaker from a noisy room: it either transcribes hiss or drops real speech. Silero decides properly, which also makes pause statistics honest — silence measured against actual speech rather than against loudness.

wav2vec2 emotion — tone the transcript cannot carry

For a speaking coach the useful signal is not the emotion label but its bearing on delivery. A tense read is what a speaker wants flagged, and listeners notice it long before they notice word choice.

ECAPA-TDNN — speaker continuity

Produces a 192-number fingerprint of a voice. The first one in a session becomes the reference; later chunks are compared by cosine similarity. Below threshold, a different person is speaking and the session says so instead of silently averaging two people together.

openSMILE — the standard prosody feature set

eGeMAPS is the accepted 88-parameter set for paralinguistics: jitter and shimmer read as vocal strain, loudness variability as dynamism. Two uses: a few are meaningful to a speaker directly, and the full vector is the natural input to a learned scoring model.

The acoustic filler detector — the one that must be trained

> **THIS IS THE STRONGEST TECHNICAL POINT IN THE PROJECT**
>
> **Whisper deletes filler words.** It was trained largely on cleaned transcripts and routinely omits "um" and "uh"; enabling its VAD filter trims the hesitation regions too. So counting fillers in the transcript under-counts *structurally* — no better regex can fix it, because the evidence is gone before the text exists. It survives only in the audio.
>
> No suitable public checkpoint exists, so the project includes a training pipeline that fine-tunes WavLM into a frame-level classifier on the **PodcastFillers** corpus. The inference wrapper and merge logic are written and tested; only the checkpoint is missing.

4 · Speech-to-Text in Depth

This is the component most likely to be questioned, and the one that was hardest to get right.

4.1 What Whisper is

An encoder–decoder Transformer trained by OpenAI on 680,000 hours of multilingual audio. Audio becomes a log-Mel spectrogram; the encoder reads it; the decoder emits text tokens autoregressively. Sizes run tiny (39M) → base (74M) → small (244M) → medium → large (1.5B). This project defaults to **base.en**, the English-only 74M variant.

4.2 Why faster-whisper and not openai-whisper

faster-whisper reimplements inference on **CTranslate2**, giving roughly 4x the speed at similar accuracy, and — critically — **int8 quantisation**. Weights stored as 8-bit integers instead of 32-bit floats cut memory about 4x and speed up CPU arithmetic substantially. This is what makes Whisper usable without a GPU.

4.3 The accuracy problem, and the fix

> **THE BUG THAT PRODUCED UNUSABLE TRANSCRIPTS**
>
> The app originally cut audio into **fixed 2.5-second slices**. Whisper was trained on **30-second windows**. Feeding it arbitrary fragments chops words in half and strips the context the decoder depends on, so the output bore little resemblance to what was said. That was the dominant cause — not the model size.

Four changes, in order of impact:

Change	Why
1. Cut on natural pauses	Audio is buffered while speaking and sent when a ~650 ms silence settles, so Whisper receives **whole utterances**. A 300 ms pre-roll keeps the first consonant
2. tiny.en → base.en	tiny mangles ordinary speech badly enough to read as broken
3. beam 1 → 5	Beam 1 is greedy decoding — fastest and least accurate
4. Decode guards	Temperature fallback plus no-speech, log-probability and compression-ratio thresholds make Whisper **decline** rather than invent text for breath and room tone

4.4 Measured result

16.8%	2.56 s	1.05×	20
WORD ERROR RATE	MEDIAN DECODE	REALTIME FACTOR	UTTERANCES TESTED

Tested against **speechocean762**, a corpus of L2-accented read English with ground-truth transcripts. WER is digit-normalised, because Whisper writing "0 3 5 1" for "ZERO THREE FIVE ONE" is correct transcription that a naive metric scores as 100% wrong.

4.5 The hybrid design

```
STT_ENGINE=auto   # Groq if keyed → OpenAI if keyed → local faster-whisper
```

Cloud engines are preferred when a key exists because whisper-large-v3 is both more accurate and faster than anything local on CPU. Every cloud path falls back to local on failure, so the feature degrades rather than breaking.

4.6 Two details worth mentioning

- **Cross-chunk context.** The tail of the previous transcript is passed as `initial_prompt`, so the decoder has context across utterance seams.
- **Inference is serialised.** `WhisperModel.transcribe` is not thread-safe, and this runs inside FastAPI's threadpool, so decoding is guarded by a lock.

5 · The Algorithms

5.1 Loudness — RMS and dBFS

```
rms  = sqrt(mean(x²))
dBFS = 20 · log₁₀(rms)
```

All thresholds are on the **dBFS** (logarithmic) scale, not linear RMS. Linear thresholds swing wildly with microphone gain and browser AGC; on a log scale the same thresholds degrade gracefully. Silence −45 dBFS, quiet −36, loud −12.

5.2 Pitch — autocorrelation with three guards

A 64 ms Hann-windowed frame is correlated with itself; the lag of the strongest peak is the period, and f0 = sample_rate ÷ lag. Naive implementations fail constantly, so three guards were added:

1. **Voiced gating** — frames below −50 dBFS are skipped. Silence has no pitch, and including it corrupts the standard deviation that drives 25% of the confidence score.
2. **Local-maximum check** — the peak must be a genuine interior maximum. Without this, a signal whose period lies outside the search range pins to a boundary: **a 20 Hz desk bump was being reported as a tense 400 Hz voice.**
3. **Octave correction** — autocorrelation peaks at every multiple of the true period, so the raw maximum often lands an octave low. Sub-multiples are preferred when at least 85% as strong.

Parabolic interpolation then refines the peak to sub-sample accuracy, and the reported mean discards outliers beyond ±60% of the median.

5.3 Resampling — polyphase at the exact ratio

```
divisor = gcd(src_rate, dst_rate)
resample_poly(audio, dst//divisor, src//divisor)
# 48000 → 16000 reduces to up=1, down=3 — exact, not approximated
```

> **A BUG WORTH DESCRIBING**
>
> The original code read the sample rate and threw it away, then assumed 16 kHz everywhere. A browser sending 48 kHz audio had its **pitch and speaking rate wrong by roughly 3×**. It only appeared to work because the frontend happened to request 16 kHz.

5.4 Clarity — MATTR, not type-token ratio

Type-token ratio (unique ÷ total words) is strongly length-dependent: a five-word chunk almost always scores 1.0, a hundred-word one rarely clears 0.6. So a per-chunk score and a whole-session score are not comparable. **MATTR** — moving-average TTR — averages TTR over a sliding 25-word window and stays stable as text grows.

```
mattr = mean(TTR(w) for w in sliding_windows(words, 25))
score = min(1, mattr / 0.75) × 100 − filler_rate × 300
```

5.5 Filler detection — context-aware, not a word list

Word	Rule	Counted	Not counted
`um`, `uh`	always	Hesitation sounds have no lexical use	
`so`	clause-initial	"**So** the answer is…"	"…**so** we shipped"
`right`	clause-final	"exactly **right**"	"the **right** answer"
`like`	lexical context	"it was **like** really fast"	"I **like** this"

5.6 Confidence score

```
score = 100 × (0.25·pace + 0.25·pitch_variation + 0.20·loudness + 0.30·fluency)
```

Each sub-score is a piecewise-linear curve mapping a measurement to 0–1.

> **BE HONEST ABOUT THIS IF ASKED**
>
> Those four weights are a **plausible prior, not a measured result**. A training script fits them against human ratings in speechocean762 and produces $R^2 = 0.20$ — but that corpus has near-constant loudness and 95% of clips contain no fillers, so two of the four weights cannot be supported by it. The script detects near-constant features and **refuses to endorse** their weights rather than printing four confident-looking numbers.

5.7 Smoothing — exponential moving average

```
ema = α · value + (1 − α) · previous      # α = 0.4
```

A score from 2.5 seconds of audio is statistically noisy — one word is 24 WPM of granularity. The EMA gives a trend line, and the UI displays it in preference to the raw per-chunk number.

5.8 Adaptive silence gate

```
gate = max(HARD_FLOOR, min(ABSOLUTE_SILENCE, p10_loudness + 8 dB))
```

Deliberately asymmetric: the absolute threshold is a **ceiling, never a floor**. A quiet microphone lowers the gate so speech still reaches the transcriber; a noisy room can never raise it and start transcribing hiss.

6 · Posture & Gesture

6.1 The architecture, and why

MediaPipe runs in the browser. Scoring runs in Python. The browser detects 33 body landmarks per frame and sends only those coordinates.

Consequence	Why it matters
Video never leaves the machine	A privacy property, not a claim — there is no code path that uploads a frame
~25 KB per 2.5 s batch	Sending video would be megabytes. Landmarks are 33 points × 4 numbers
Coaching logic stays in Python	Same place as the speech coaching, so the two can be fused

6.2 What is measured

Metric	How
Shoulder tilt	Angle of the shoulder line from horizontal
Head tilt	Angle of the ear-to-ear line
Torso lean	Angle of hip-midpoint → shoulder-midpoint from vertical
Openness	Shoulder width ÷ torso height — collapses when hunched
Forward head	Ear depth relative to shoulder depth — the "screen reading" posture
Gesture rate	Runs of wrist motion above threshold, per minute
Sway	Standard deviation of torso centre, in shoulder-widths
Fidget ratio	Fraction of frames with small persistent motion that never becomes a gesture

Every distance is divided by shoulder width, so nothing changes when the speaker moves nearer to or further from the camera.

> **THE SUBTLE DETAIL — WORTH RAISING UNPROMPTED**
>
> MediaPipe normalises x and y to 0–1 **independently**, against a frame that is almost never square. Computing an angle from those raw values stretches it by the aspect ratio, so **level shoulders in a 16:9 frame measure as tilted**. Every angle here is computed in aspect-corrected space, and there is a regression test pinning it.

6.3 Two scoring rules

- **Never score what was not seen.** If the hips are out of frame, torso lean is unknown — reporting a confident 0 would be a lie. Weights renormalise over the visible dimensions, and the response lists which ones counted.
- **Gestures are counted as runs, not frames.** One sweeping arm movement is one gesture, not fifteen.

6.4 Presence — multimodal fusion

```
presence = 0.6 · voice_score + 0.4 · body_score
```

If either half is missing the other stands alone — a speaker with no camera must not appear to score 50%.

7 · Module Map

35 Python modules, ~5,300 lines.

Module	Lines	Responsibility
Application		
`main.py`	237	App construction, CORS, error handlers, health, static mounts, model warm-up
`middleware.py`	196	Request IDs, token-bucket rate limiting, optional API key
`schemas.py`	276	Every response as a Pydantic model
`routes/analyze.py`	507	All ten endpoints and the pipeline orchestration
Audio & speech		
`services/audio_io.py`	118	Decode → mono → float32 → resample → sanitise
`services/audio_analysis.py`	258	Energy, pitch, pause detection
`services/speech_to_text.py`	295	Hybrid cloud/local STT, inference lock, decode guards
Language		
`services/filler_detection.py`	117	Context-aware filler scanning
`services/keyword_detection.py`	57	Longest-match keyword n-grams
`services/clarity_analysis.py`	70	MATTR clarity scoring
`utils/text.py`	84	Shared tokeniser with clause awareness
Scoring & state		
`services/confidence_score.py`	149	Weighted composite scoring
`services/feedback_engine.py`	218	Threshold rules → coaching messages
`services/llm_feedback.py`	188	Throttled LLM insight, with timeout and cost cap
`services/session_store.py`	392	Session state, smoothing, adaptive gate, speaker continuity, posture trend
Posture		
`services/pose_analysis.py`	291	Landmark geometry — angles, ratios, aspect correction
`services/gesture_analysis.py`	182	Movement over time — gestures, sway, fidget
`services/posture_feedback.py`	228	Posture score, coaching copy, presence fusion
Optional ML (9 modules, ~1,500 lines)		
`services/ml/registry.py`	209	Lazy loading, device resolution, per-model status and errors
`services/ml/enrichment.py`	146	Orchestrates whichever models are enabled
`ml/pitch · vad · emotion · speaker · prosody · alignment · disfluency`	~1,100	One module per model, all following the same optional contract
Configuration		
`utils/config.py`	373	Single source of truth — ~50 environment settings, typed and validated
`utils/helpers.py`	227	Thresholds, dBFS conversion, logger

8 · API Surface

Method	Endpoint	Purpose
POST	`/api/analyze`	Analyse one audio utterance. Returns metrics, transcript, score, coaching
POST	`/api/pose`	Score a batch of body landmarks for posture and gesture
POST	`/api/diarize`	Segment a full recording by speaker
POST	`/api/session`	Open a session
GET	`/api/session/{id}`	Rolling totals and smoothed averages
GET	`/api/session/{id}/report`	Full report with stitched transcript
DELETE	`/api/session/{id}`	Close a session, return the final report
GET	`/api/status`	Lightweight readiness probe
GET	`/api/health`	Deep check — engine, model load state, LLM, sessions, ML registry
GET	`/docs`	Interactive OpenAPI docs, generated from the Pydantic models

8.1 Degraded responses

A failed transcription does not fabricate a score. Acoustic metrics remain genuine, so they are returned and the response is flagged.

Warning code	Meaning
`silence_skipped`	Below the gate; STT deliberately not run
`stt_unavailable`	Every transcription engine failed
`confidence_acoustic_only`	Pace and fluency sub-scores are not evidence-backed
`speaker_changed`	Voice no longer matches the session reference
`partially_out_of_frame`	Body seen in too few frames to score responsibly

9 · Design Decisions

Decision	Alternatives	Reasoning
FastAPI	Flask, Django, Express	Native async, Pydantic validation, automatic OpenAPI
soundfile + scipy	librosa, ffmpeg, pydub	librosa drags in numba and ~50 MB for functions replaced by ~250 lines of NumPy. ffmpeg is a system dependency
faster-whisper	openai-whisper, Vosk, wav2vec2	~4x faster at similar accuracy, and int8 makes CPU inference viable
Browser capture	Server-side mic/camera	A deployed server has neither. Also keeps video and audio on the user's machine
Landmarks not video	Uploading frames	25 KB vs megabytes, and video never leaves the machine
In-process sessions	Redis, Postgres	Sessions are ephemeral coaching state. A database is right only when history must outlive the process
All ML off by default	Bundled and always on	Base install must stay ~1 GB and must work without torch
Pydantic schemas	Raw dicts	Untyped dicts are how the docs drifted out of sync with the routes

9.1 Rejected, and why

- **Fixed-interval audio chunks** — chopped words mid-syllable and destroyed accuracy. Replaced with pause-based segmentation.
- **Audio overlap between chunks** — would duplicate words at seams. Decoder prompt carryover achieves the benefit without the deduplication problem.
- **Summing acoustic and text filler counts** — double-counts the same event when Whisper does keep an "um". The maximum per label is used instead.
- **Root TTR for clarity** — length-normalises in the wrong direction for short chunks; MATTR chosen instead.
- **Serverless deployment** — 324 MB against a 250 MB limit, and statelessness would destroy the session layer.

10 · Numbers to Quote

All measured on this machine — Windows 11, CPU only, no NVIDIA GPU — not estimated.

10.1 Scale

35	~5,300	196	10
BACKEND MODULES	LINES OF PYTHON	TESTS PASSING	ENDPOINTS

10.2 Speech recognition

Measurement	Value
Word error rate (digit-normalised)	**16.8%** over 20 utterances of L2-accented read speech
Median decode time	2.56 s
Realtime factor	1.05× (decoding takes about as long as the audio)

10.3 Model latency (2.5 s chunk budget)

Component	Warm cost	Share of budget
DSP baseline (no ML)	~90 ms	4%
CREPE pitch (tiny)	~780 ms	31%
Silero VAD	~270 ms	11%
Emotion (wav2vec2)	~385 ms	15%
Speaker embedding (ECAPA)	~375 ms	15%
openSMILE eGeMAPS	~1600 ms	64%
All together	**~3400 ms**	**136% — over budget**

10.4 Dependency footprint

Bundle	Size
FastAPI + NumPy + SciPy + SoundFile	216 MB
+ faster-whisper + CTranslate2 + onnxruntime	324 MB
+ PyTorch (optional ML layer)	+497 MB

10.5 Bugs found and fixed

Sixteen, each with a regression test. The four most worth describing:

Bug	Effect
Sample rate read then discarded	48 kHz input → pitch and speaking rate wrong by ~3×
Fixed 2.5 s audio slices	Transcripts bore little resemblance to the speech
Pitch peak pinned to search boundary	A 20 Hz desk bump reported as a tense 400 Hz voice
Emotion labels only long-spelled	Audio 98% classified angry scored tension 0.003

11 · Likely Questions

Q Why not just use a speech-to-text API and be done?

> Transcription is one of eleven measurements. The project is about *how* something was said — pace, loudness, vocal variety, pauses, fluency, posture — and none of that is in a transcript. Whisper supplies the words; the rest is signal processing and scoring.

Q Why write your own pitch tracker instead of using a library?

> librosa provides one, but it pulls in numba and ~50 MB for a function that is about 60 lines of NumPy and SciPy. Writing it also meant the three failure modes could be handled explicitly — voiced gating, boundary peaks, octave errors — which is how the 20 Hz bug was found. CREPE is available as the neural alternative when accuracy matters more than weight.

Q Is this real-time?

> Near real-time. Audio is segmented on natural pauses, so feedback arrives shortly after you stop speaking — roughly one second of decoding per second of speech on CPU. It is not streaming word-by-word; that would need a WebSocket transport and a streaming model.

Q How accurate is the transcription, honestly?

> 16.8% word error rate on accented read speech with the local base.en model. Cleaner native speech does better. A Groq API key switches to whisper-large-v3, which is substantially more accurate and faster — the architecture supports both.

Q Is the confidence score meaningful?

> The *sub-scores* are well grounded — each maps a real measurement through a documented curve. The *weights* combining them are a plausible prior, not a fitted result. There is a training script that fits them against human ratings, and it correctly reports that the available corpus can only support two of the four. That honesty is deliberate.

Q Does the video get uploaded?

> No. MediaPipe runs in the browser and only 33 landmark coordinates are sent — about 25 KB per batch. There is no code path that uploads a frame.

Q What happens if a model fails to load?

> The feature turns itself off and reports why. Every optional model sits behind a registry that records the load error, surfaces it on `/api/health`, and never retries — so a broken dependency costs one startup attempt, not one per request. The DSP path stays wired as the fallback.

Q Why is the whole ML layer disabled by default?

> Because `pip install -r requirements.txt` must produce a working app. PyTorch alone is 497 MB. Enabling models is a deliberate choice with a measured time cost, and the budget only fits about two of them on CPU.

Q Why can this not go on Vercel?

> Two reasons. Size: 324 MB against a 250 MB serverless limit. And state: serverless functions are stateless, so the session store — smoothing, cumulative totals, adaptive gate, decoder context — would be destroyed between invocations. The frontend goes on Vercel; the backend needs a host that keeps a process alive.

Q What is the weakest part?

> Three things. The confidence weights are unfitted. The acoustic filler detector has no trained checkpoint, so filler counts still under-report whatever Whisper deleted. And it is English-only. All three are documented rather than hidden.

Q What would you do next?

> Train the filler detector on PodcastFillers — it fixes a structural blind spot, not a tuning issue. Then fit the confidence weights on a corpus of rated spontaneous speech. Then a WebSocket transport to remove the latency floor.

DOCUMENT PROVENANCE

Every version number, line count, latency, and error rate here was read or measured from the running system rather than recalled. Dependency versions come from the installed environment; latencies were timed after model warm-up; the word error rate was computed against ground-truth transcripts from speechocean762.